

Reinforcement Learning Project: Simulating Cricket Match using Inverse Reinforcement Learning



Name: Aadil Mohammad Husain

Roll Number: MT2024001

Name: Shaikh Mohammad Irfan

Roll Number: MT2024138

Name: Dora Bhavya Rishitha

Roll Number: MT2024045

Table of Contents-

Introduction-	3
Dataset Used-.....	3
Inverse Reinforcement Learning	Error! Bookmark not defined.
Tools Used-.....	3
Steps Involved-	4
1. Data Extraction-.....	4
2. Feature Engineering and Trajectory Preparation-	4
3. Visualizing the Learned Coefficients-	7
4. Computing State Transition and Wicket Probabilities-	8
5. Building the Optimal Policy from Learned Rewards-	9
6. Simulating a Full Innings with Epsilon-Greedy Strategy –.....	10
7. Optimizing Policy with Dynamic Risk Awareness –	11

Introduction-

For our Reinforcement Learning project, we explored the strategic depth of one-day international cricket by modeling the game as a Markov Decision Process. Our approach involved using both forward and inverse reinforcement learning techniques to understand and optimize gameplay decisions. We trained a value function to estimate the scoring potential of different game situations and compared its effectiveness to traditional methods used in professional cricket.

Additionally, by analysing match-winning performances, we derived a reward model that captures expert strategies, allowing us to simulate and evaluate alternative gameplay policies. This project not only offers a data-driven perspective on decision-making in cricket but also demonstrates how reinforcement learning can be applied to turn-based sports for performance analysis and strategic improvement.

Dataset Used-

We sourced our dataset from **Cricsheet**, an open-access platform that provides detailed ball-by-ball data for various cricket formats. The dataset includes comprehensive match information such as player names, runs scored, wickets taken, and over-by-over progression. For our project, we specifically used one-day international (ODI) match data to model the game dynamics. It consists of nearly 2.9k matches trajectories. The rich granularity of Cricsheet's data allowed us to construct accurate state representations and reward structures necessary for implementing reinforcement learning algorithms.

Link to Dataset-

<https://cricsheet.org/matches/>

Tools Used-

To implement this project, we primarily used Python due to its rich ecosystem of machine learning libraries and ease of handling data. We conducted our experiments and model development within Jupyter Notebook, which provided an interactive environment ideal for iterative testing, data visualization, and debugging. Libraries such as NumPy and Pandas were used for data manipulation, while Matplotlib and Seaborn supported visual analysis.

Technique Used-

Algorithm : Inverse Reinforcement Learning (IRL) algorithm used

Result: Coefficients or weights for each feature

- Define a set of features;
- Compute the expert value function for each feature;
- Initialize all coefficients as per the domain(cricket) knowledge;
while *There are non-expert trajectories remaining* **do**
 - Compute the non-expert value function for each feature using the set coefficients;
 - Solve the optimization problem using linear programming based on pool of conditions plus a new condition;
 - Update coefficients based on optimization results
 - Add the conditions to the pool of conditions**end**

Steps Involved-

1. Data Extraction-

To extract meaningful information from raw cricket match data, we used a dataset of one-day international (ODI) matches sourced from **Cricsheet**, which provides match data in YAML format. We uploaded and extracted the data

The YAML files were parsed using Python's yaml library, and we iterated through each delivery in every inning to extract key features such as runs scored, number of wickets, ball number, batting and bowling teams, wides, no-balls and to ensure only legal deliveries were processed when needed. The extracted data was stored in DataFrame, with cumulative statistics such as total runs and wickets being calculated per delivery. Finally, the processed dataset was saved as a CSV file (odi_parsed.csv).

100% | ██████████ | 2929/2929 [21:22:00:00, 2.28it/s]
 Saved to odi_parsed.csv

	match_id	inning	team	over	ball_in_over	delivery_index	extras	score_band	wickets_lost	is_legal	runs_batsman	is_wicket	batsman	non_striker	bowler	cumulative_runs
0	209582	1	Pakistan	0	1	0	0	0	0	True	0	0	Salman Butt	Shahid Afridi	IDR Bradshaw	0
1	209582	1	Pakistan	0	2	1	0	0	0	True	0	0	Salman Butt	Shahid Afridi	IDR Bradshaw	0
2	209582	1	Pakistan	0	3	2	0	0	0	True	0	0	Salman Butt	Shahid Afridi	IDR Bradshaw	0
3	209582	1	Pakistan	0	4	3	0	0	0	True	0	0	Salman Butt	Shahid Afridi	IDR Bradshaw	0
4	209582	1	Pakistan	0	5	4	0	0	0	True	0	0	Salman Butt	Shahid Afridi	IDR Bradshaw	0

STATE_SPACE:

[over, wickets_lost, score_band, ball_in_over, extras]

Score_band is score//10 (to avoid variance)

ACTION_SPACE:

[0,1,2,3,4,6]

METHOD_1:

In this project, we address the challenge of modeling cricket match scenarios where the data may not strictly follow the Markov property. To handle this, we adopt a **Monte Carlo (MC) learning approach**, known for its robustness to such deviations. Since the state space is large, we apply **value function approximation** using a **neural network** to estimate the percentage of scoring resources left at any game state.

The dataset is split into **90% training and 10% testing**, and **10-fold cross-validation** is used to validate the model. For evaluation, a random interruption point is introduced after the 20th over of an innings in the test set. At this point, the model predicts the value function (resources left), which is compared against the **Duckworth-Lewis-Stern (DLS)** method, the traditional gold standard.

The **predicted final score** is computed using:

$$\text{predictedFinalScore} = [\text{ScoreAtInterruption} / (1-r)]$$

where r is the resources-left percentage from either the MC model or the DLS table.

The **percentage error** between predicted and actual final scores is calculated for each game:

$$\%Error = (\text{Predicted Score} - \text{Actual Score}) / \text{Actual Score} \times 100$$

This error is averaged across the test games and folds, and the standard deviation is also reported to measure consistency. The MC-based model's performance is then compared to that of the DLS method.

A **neural network** is used as the function approximator, with an input layer of 5 neurons (for first innings) or 6 neurons (for second innings), several hidden layers optimized for performance, and a single output neuron representing the value function.

This system can serve as a **direct replacement for the DLS method**, and can be easily implemented in software. It also has the flexibility to be extended for other objectives, such as predicting win probabilities instead of scoring potential.

METHOD_2:

2. Feature Engineering and Trajectory Preparation-

This project applies Inverse Reinforcement Learning (IRL) to learn a reward function that differentiates expert from non-expert trajectories. The core objective is to extract trajectories from matches where the final outcome (win or loss) is used to label each team's first innings as expert (winning team) or non-expert (losing team). The `prepare_features` function is responsible for this extraction. It first ensures that each match has data from both innings and then identifies the winner based on total cumulative runs. For each team, only their first innings is used to generate a trajectory consisting of state-action pairs, which are constructed using the `process_trajectory` function. Each state is represented by five features: the over number, wickets lost, score band (in 10-run intervals), the ball number within the over, and a binary indicator for extras. The actions are defined as the runs scored by the batsman on each delivery, restricted to {1, 2, 3, 4, 6} to reflect meaningful run choices.

To prepare the data for IRL, the function `calculate_feature_expectations` is used to compute normalized average feature vectors from the trajectories. State features are scaled to a $[0,1]$ range to ensure comparability, while action features are treated as binary indicators, with their contribution weighted by the number of runs. This produces a 10-dimensional feature vector—five dimensions for state features and five for action frequencies.

The learning process is carried out by the `inverse_reinforcement_learning` function. It begins with an initial set of reward function coefficients based on domain knowledge. In each iteration, the algorithm compares the feature expectation of a non-expert trajectory with that of the expert. If the expert's margin of superiority is below a threshold, a new constraint is added to a growing set of linear inequalities. These inequalities are then used to solve a linear program (LP) using `scipy.optimize.linprog`, which finds a new set of reward weights that maintain the minimum required margin. The LP includes bounds on each weight (e.g., wickets are penalized) and incorporates L2 regularization through dummy variables to prevent overfitting or extreme values. The updated reward vector is normalized to ensure stability and interpretability.



```
Preparing features...

Found 1452 expert and 1401 non-expert trajectories

Running IRL...
Normalized expert feature expectations:
State: [0.54546712 0.26465272 0.28012319 0.60021934 0.00390941]
Actions: [0.001      0.0005      0.00033333 0.00025      0.00016667]

=== Learned Reward Function ===
State coefficients:
  Over progression (x1): 0.6786
  Wickets lost (x2): -0.2545
  Score band (x3): 0.5937
  ball in over (x4): 0.3393
  extras (x5): 0.0848

Action coefficients (weighted by 1000):
  1 run (y1): 0.2545
  2 runs (y2): 0.6786
  3 runs (y3): 0.0848
  4 runs (y4): 0.5089
  6 runs (y6): 0.3393
```

DOMAIN_KNOWLEDGE:

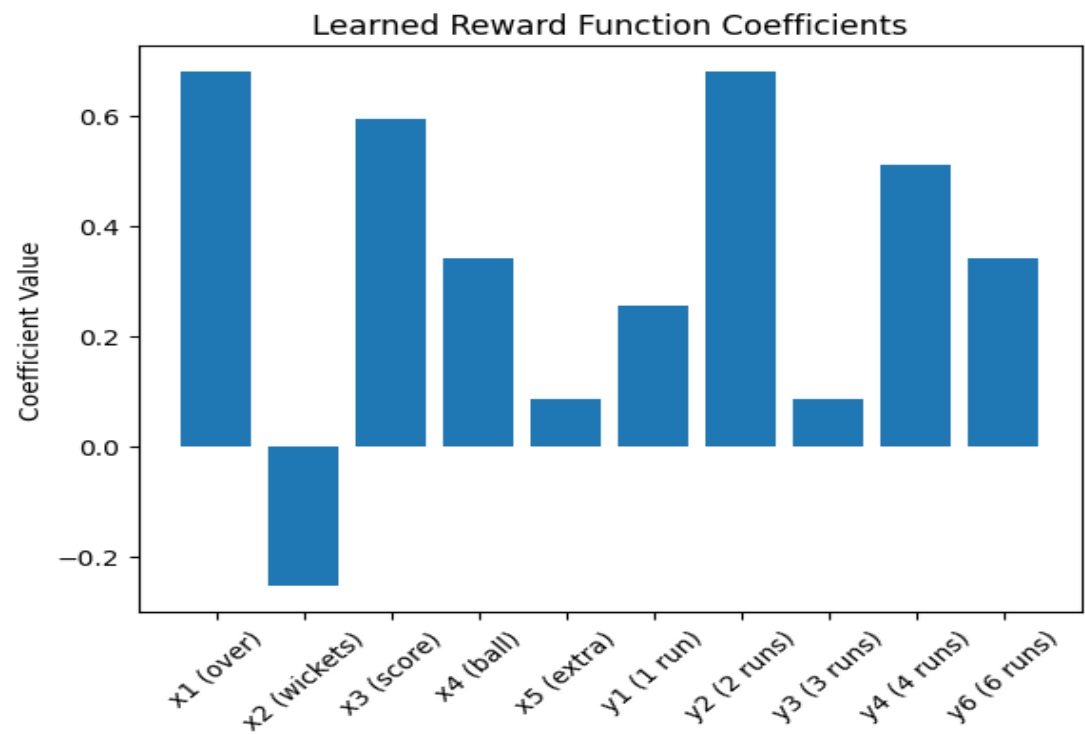
In the beginning overs play aggressively. In middle overs play more runs not boundaries and in the final overs go for shots, but more vulnerable to wickets in final overs. If we hit a 6 or 3, it's more risk for a wicket. Hitting 4's and 2's is the most safest. And many euristics like this .

Finally, the learned coefficients—representing the relative importance of state and action features in expert decision-making—are printed. Additionally, the expert and non-expert trajectories are serialized and saved using Python’s pickle module for potential reuse in future analyses or for training decision-making policies. This approach provides a data-driven way to uncover strategic preferences in cricket gameplay and paves the way for applications in simulation, coaching, or automated strategy generation.

Expert and Non-Expert trajectories saved successfully to /content/drive/MyDrive/cricket_IRL_trajectories_1st_innings

3. Visualizing the Learned Coefficients (Reward Function)-

To visualize the learned reward function, a bar plot is generated using the plot_coefficients function. This function takes the final reward vector and plots the value of each coefficient to provide an intuitive understanding of the model’s learned preferences. The ten coefficients are labeled to reflect the features they correspond to—five state-related (over number, wickets lost, score band, ball number within the over, and extras) and five action-related (runs scored: 1, 2, 3, 4, and 6). Using Matplotlib, a bar chart is created with feature labels on the x-axis and coefficient values on the y-axis. The bars visually represent the relative importance of each feature in the learned reward function. For example, if the coefficient for “x2 (wickets)” is strongly negative, it implies that the IRL model has learned to penalize loss of wickets. This visualization aids in interpreting the reward structure and validating whether the learned weights align with cricketing intuition and strategy.



4. Computing State Transition and Wicket Probabilities-

The function `calculate_transition_and_wicket_probs`, which calculates empirical probabilities for both scoring runs and losing wickets from each game state. The analysis is restricted to the **first innings** data and sorts the delivery records by match ID, over number, and ball number. A key correction implemented here is the **resetting of cumulative runs and wickets** when a new match begins—this prevents data from different matches from being mixed during trajectory tracking.

Each state is defined as a 5-tuple: (over number, cumulative wickets lost, score band, ball number in the over, extras indicator). For every delivery, the code updates `ball_counts` to track how many times a state was encountered, `wicket_counts` to count how often a wicket fell from that state, and `run_counts` to record the number of times a specific run value was scored from that state (excluding deliveries where a wicket fell). The cumulative score and wicket count are updated accordingly after each ball to ensure accurate state representation.

After all deliveries are processed, the function computes **wicket probabilities** by dividing the number of times a wicket fell in a state by the total times that state was encountered. **Transition probabilities** for runs (1, 2, 3, 4, 6) are calculated by dividing the frequency of each run value by the total number of non-wicket deliveries from that state. The output is a dictionary of transition probabilities (`transition_probs`) and a separate dictionary of wicket probabilities (`wicket_probs`), both of which model the stochastic environment in which the IRL agent operates. These probability models are crucial for simulating agent behavior and evaluating policies under realistic cricket match conditions.

```
[ ] trans_probs
```

```
{(0, 0, 0, 1, 1): {1: 0.8730650154798761,  
 5: 0.034055727554179564,  
 2: 0.06191950464396285,  
 4: 0.02476780185758514,  
 3: 0.006191950464396285},  
(0, 0, 0, 2, 0): {0: 0.7666919575113809,  
 1: 0.1286039453717754,  
 4: 0.0637329286798179,  
 2: 0.032625189681335355,  
 3: 0.006449165402124431,  
 6: 0.0015174506828528073,  
 5: 0.00037936267071320183},  
(0, 0, 0, 3, 0): {0: 0.7571761055081458,  
 2: 0.02909231962761831,  
 6: 0.0023273855702094647,  
 4: 0.06594259115593483,  
 1: 0.1330488750969744,  
 5: 0.0003878975950349108,  
 3: 0.012024825446082235},  
(0, 0, 0, 4, 0): {1: 0.13205282112845138,  
 0: 0.7474989995998399,  
 4: 0.07202881152460984,  
 3: 0.0056022408963585435,  
 2: 0.04081632653061224,  
 6: 0.0020008003201280513},
```



```
wicket_probs
{(0, 0, 0, 1, 1): 0.0,
 (0, 0, 0, 2, 0): 0.02225519287833828,
 (0, 0, 0, 3, 0): 0.020516717325227963,
 (0, 0, 0, 4, 0): 0.026110678098207326,
 (0, 0, 0, 5, 0): 0.02133544053733702,
 (0, 0, 0, 6, 0): 0.02622950819672131,
 (1, 0, 0, 1, 1): 0.0,
 (1, 0, 0, 2, 0): 0.01990740740740741,
 (1, 0, 0, 3, 0): 0.02370585389453314,
 (1, 0, 0, 4, 0): 0.021354166666666667,
 (1, 0, 0, 5, 0): 0.017260579064587972,
 (1, 0, 0, 6, 1): 0.0,
 (1, 0, 0, 7, 0): 0.040214477211796246,
 (1, 0, 0, 8, 0): 0.025974025974025976,
 (2, 0, 0, 1, 0): 0.019404019404019403,
 (2, 0, 0, 2, 0): 0.02318623784592371,
 (2, 0, 0, 3, 0): 0.016060862214708368,
 (2, 0, 1, 4, 0): 0.018018018018018018,
 (2, 0, 1, 5, 0): 0.023579849946409433,
 (2, 1, 1, 6, 0): 0.014760147601476014,
 (3, 1, 1, 1, 0): 0.03741496598639456,
```

5. Building the Optimal Policy from Learned Rewards-

The function `build_optimal_policy` constructs an optimal policy by determining the best possible action (i.e., number of runs to aim for) for each game state, using the reward function previously learned through inverse reinforcement learning (IRL). The policy serves as a mapping from states—defined by match context—to actions, with the objective of maximizing expected immediate reward.

Each state is evaluated across a set of **possible actions**: [0, 1, 2, 3, 4, 6], corresponding to the possible number of runs that can be scored off a single ball. For each action, a **feature vector** of size 10 is constructed, combining both **state-related features** (such as over number, wickets lost, score band, ball number, and presence of extras) and **action-specific features** encoded as one-hot vectors.

The first five components of the vector represent normalized state attributes:

- x1: Over number (normalized by 50)
- x2: Wickets lost (normalized by 10)
- x3: Score band (normalized by 40)
- x4: Ball number in the over (normalized by 6)
- x5: Extras flag (0 or 1)

The remaining five components represent one-hot encodings of the action taken:

- y1 through y6: One-hot activation for the chosen action (e.g., scoring 1 run, 2 runs, etc.)

For each action, the **reward** is computed by taking the dot product between the learned IRL coefficients and the feature vector. Since the focus here is on **immediate rewards**, the action with the highest reward value is selected as the optimal decision for that state. This process is repeated for every state present in the `transition_probs` dictionary, and the results are stored in the `optimal_policy` dictionary.

The final output is a complete state-to-action mapping that represents the optimal policy based on the inferred preferences of expert players. This policy can be used to simulate or analyze agent behavior under realistic match scenarios.

```
[ ] optimal_policy
➡ {(0, 0, 0, 1, 1): 4,
    (0, 0, 0, 2, 0): 4,
    (0, 0, 0, 3, 0): 4,
    (0, 0, 0, 4, 0): 4,
    (0, 0, 0, 5, 0): 4,
    (0, 0, 0, 6, 0): 4,
    (1, 0, 0, 1, 1): 4,
    (1, 0, 0, 2, 0): 4,
    (1, 0, 0, 3, 0): 4,
    (1, 0, 0, 4, 0): 4,
    (1, 0, 0, 5, 0): 4,
    (1, 0, 0, 6, 1): 4,
    (1, 0, 0, 7, 0): 4,
    (1, 0, 0, 8, 0): 4,
```

In practice, most of the states in the project are either impossible or meaningless. For example, a scenario where only 10 runs are scored in the 48th over is unlikely, a match where 9 wickets are lost in the second over is nonsensical, and a situation where 10 wickets are lost but the team is still playing is illegal. Thus, only a small subset of the states are realistic. The `transition_probs` dictionary already contains the realistic visited states, based on real historic matches. Therefore, it is both efficient and correct to only solve for the states in `transition_probs`.

6. Simulating a Full Innings with Q-learning Epsilon-Greedy Strategy –

In this step, we simulated a full cricket innings using an epsilon-greedy strategy to decide the batsman's actions. The simulation starts from an initial game state with zero overs, zero wickets, and zero runs, and progresses ball by ball until either 50 overs are completed or 10 wickets fall. At each step, the optimal action for the current state is retrieved from a precomputed policy. However, instead of always following the optimal action, the epsilon-greedy strategy is applied—meaning there's a small probability (epsilon) that a random action is chosen from the list of possible run values (0, 1, 2, 3, 4, 6) to encourage exploration. The simulation also checks if a wicket falls, based on a probability lookup, and updates the state accordingly. If no wicket falls, the selected action adds runs to the total, updates the score band, and advances the ball count. Once six balls are bowled, the over count is incremented. This process continues until the innings ends, with the final total runs representing the outcome of the simulated innings under the given policy.

The final score of the inning is given by-

```
[31, 9, 0, 5, 0] --> 232 action is 0
[32, 9, 0, 0, 0] --> 232 action is 0
[32, 9, 0, 1, 0] --> 232 action is 0
[32, 9, 0, 2, 0] --> 232 action is 0
[32, 9, 0, 3, 0] --> 232 action is 0
[32, 9, 0, 4, 0] --> 232 action is 0
```

```
[32, 9, 0, 5, 0] --> 232 action is 0
[33, 9, 0, 0, 0] --> 232 action is 0
[33, 9, 0, 1, 0] --> 232 action is 0
[33, 9, 0, 2, 0] --> 232 action is 0
[33, 9, 0, 3, 0] --> 232 action is 0
[33, 9, 0, 4, 0] --> 232 action is 0
[33, 9, 0, 5, 0] --> 232 action is 0
[34, 9, 0, 0, 0] --> 233 action is 1
[34, 9, 0, 1, 0] --> 239 action is 6
[34, 9, 0, 2, 0] --> 239 action is 0
[34, 9, 0, 3, 0] --> 239 action is 0
[34, 9, 0, 4, 0] --> 239 action is 0
[34, 10, 0, 5, 0] --> 239 action is 0
```

Simulated Final Score: 239 runs

The model exhibits several behavioral flaws due to limitations in its reward structure and strategic understanding. It tends to favor boundaries excessively because the reward coefficients are skewed toward 4s and 6s, making the model believe these are always the best choices. This results in overly aggressive play throughout the innings. Moreover, the model lacks dynamic match situation awareness—it doesn't differentiate between phases of the game where different strategies are needed, such as playing cautiously during the early overs and attacking aggressively in the death overs.

The absence of a sufficient penalty for risky behavior early on leads the model to take unnecessary risks right from the first ball. Additionally, it fails to reward aggressive play in the final overs, when such behavior is actually desirable. Lastly, although an epsilon-greedy mechanism is in place to introduce randomness, its influence is too weak to significantly alter the model's biased decision patterns.

7. Optimizing Policy with Dynamic Risk Awareness –

In this step, we enhanced the action-selection strategy by introducing dynamic risk awareness based on the match situation. The function evaluates all possible run-scoring actions (1, 2, 3, 4, 6) and adjusts the probability of losing a wicket depending on the number of wickets left and the overs remaining.

For instance, if the team has very few wickets in hand, the perceived risk is increased to encourage safer play. On the other hand, if it's the final phase of the innings with many wickets remaining, the model reduces the perceived risk to promote aggressive scoring. Each action is evaluated by constructing a feature vector that includes normalized state features and a one-hot encoding of the action. A raw reward is computed using a set of coefficients, and then adjusted by subtracting a penalty proportional to the dynamically adjusted wicket loss probability. The action with the highest expected reward is selected, ensuring that the model makes smarter, context-aware decisions throughout the innings.

We outlined a dynamic batting strategy based on the evolving match situation to guide intelligent shot selection.

- **Many wickets left (≥ 6) and few overs left (≤ 10):**
Play **very aggressively**, prioritizing boundaries like 4s and 6s to maximize scoring in limited time.
- **Few wickets left (≤ 3):**
Play **very safely**, preferring singles and doubles (1s and 2s), and strictly avoid risky shots.
- **Middle overs (11–35) with 4–6 wickets left:**
Adopt a **balanced strategy**, combining 2s and 4s with occasional calculated risks.
- **Early overs (0–10):**
Start **conservatively**, focusing on safe shots (1s and 2s) unless the field setting encourages attack.
- **Near end (45–50 overs), even with 4–5 wickets left:**
Increase risk-taking, as it's acceptable to lose wickets for quick runs during the final overs.
- **Extras present (e.g., no-balls, wides):**
Can justify **more aggressive play**, especially if it results in a free hit (optional, advanced tactic).

SIMULATE TRAJECTORIES FROM A STATE OF GAME:

Simulate the trajectories of the game, nearly 10 trajectories and do the average over those 10 trajectories.

```

compare_dls_vs_irl(
  over=30,
  wickets=5,
  score_band=15,
  ball_in_over=3,
  extras_flag=0,
  runs_made_till_now=152,
  optimal_policy=optimal_policy,
  transition_probs=trans_probs,
  wicket_probs=wicket_probs
)

```

 --- Match State ---
 Overs: 30, Wickets: 5, Runs so far: 152

 DLS Projected Score: 244
 IRL Policy Avg Score (10 sims): 330.90